HW 4.3: Reproducible Data Analysis

Suhas Rakothu

2026-04-16

Table of contents

1 Busiest Airports Analysis

Airports serve as a window into global economic activity, international connectivity, and recovery from disruption. This section examines six of the world's busiest passenger airports — Hartsfield-Jackson Atlanta (ATL), Frankfurt (FRA), Beijing Daxing (PKX), John F. Kennedy (JFK), Tokyo

Haneda (HND), and Dubai International (DXB) — over the period from 2020 to 2024. These years capture the dramatic collapse in air travel at the onset of COVID-19 and the equally dramatic, if uneven, recovery that followed.

1.1 Data

The passenger traffic figures below were drawn from Wikipedia's *List of Busiest Airports by Passenger Traffic* and entered manually into a tidy data frame. Each row represents one airport-year combination for a total of 36 observations. Passenger counts reflect total annual passengers (enplaned plus deplaned). Some values are unavailable: PKX has no data for 2020–2021 because the airport only opened in September 2019 and released limited early figures, and 2025 data were not yet published for any airport at the time of this analysis.

1.2 Passenger Traffic Table

The table below arranges airports as rows and years as columns, making it easy to track each airport's absolute passenger volume and to compare airports within any single year. Airports are sorted by their 2024 total in descending order so the busiest airports appear at the top.

Table 1: Annual Passenger Traffic (Millions) at Six Major Airports, 2020–2025

		Annual Passengers (Millions)					
Airport	Code	2020	2021	2022	2023	2024	2025
Hartsfield-Jackson Atlanta International	ATL	42.92	75.70	93.70	104.65	108.07	NA
Dubai International Airport	DXB	25.90	29.10	66.10	86.99	95.20	NA
Tokyo Haneda Airport	HND	25.90	16.20	49.80	85.90	91.43	NA
John F. Kennedy International Airport	JFK	16.60	30.80	55.30	62.46	63.27	NA
Frankfurt Airport	FRA	18.77	24.81	48.92	59.36	61.56	NA
Beijing Daxing International Airport	PKX	NA	NA	10.27	39.36	49.44	NA

Source: Wikipedia – List of Busiest Airports by Passenger Traffic. – indicates data not available.

The table makes several patterns immediately legible. ATL's dominance is striking: even in its worst pandemic year (2020), it still processed roughly 43 million passengers — more than double Frankfurt's volume that same year. By 2024, ATL was handling over 108 million passengers annually, a figure so large it is difficult to appreciate without comparison. DXB and HND both climbed from around 26 million in 2020 to the 91–95 million range by 2024, while JFK and FRA tracked each other closely, settling near 62–63 million. PKX, which only began appearing in the data in 2022 at just over 10 million passengers, grew to nearly 50 million by 2024 — a fivefold increase in two years that reflects both its relative youth and China's relaxation of COVID-era travel restrictions.

1.3 Passenger Traffic Over Time

While the table is effective for precise comparisons at a given year, a line plot reveals the shape of each airport's recovery trajectory. The panel below plots passenger traffic in millions from 2020

through 2024, with each airport assigned a distinct color and labeled directly at its final data point.

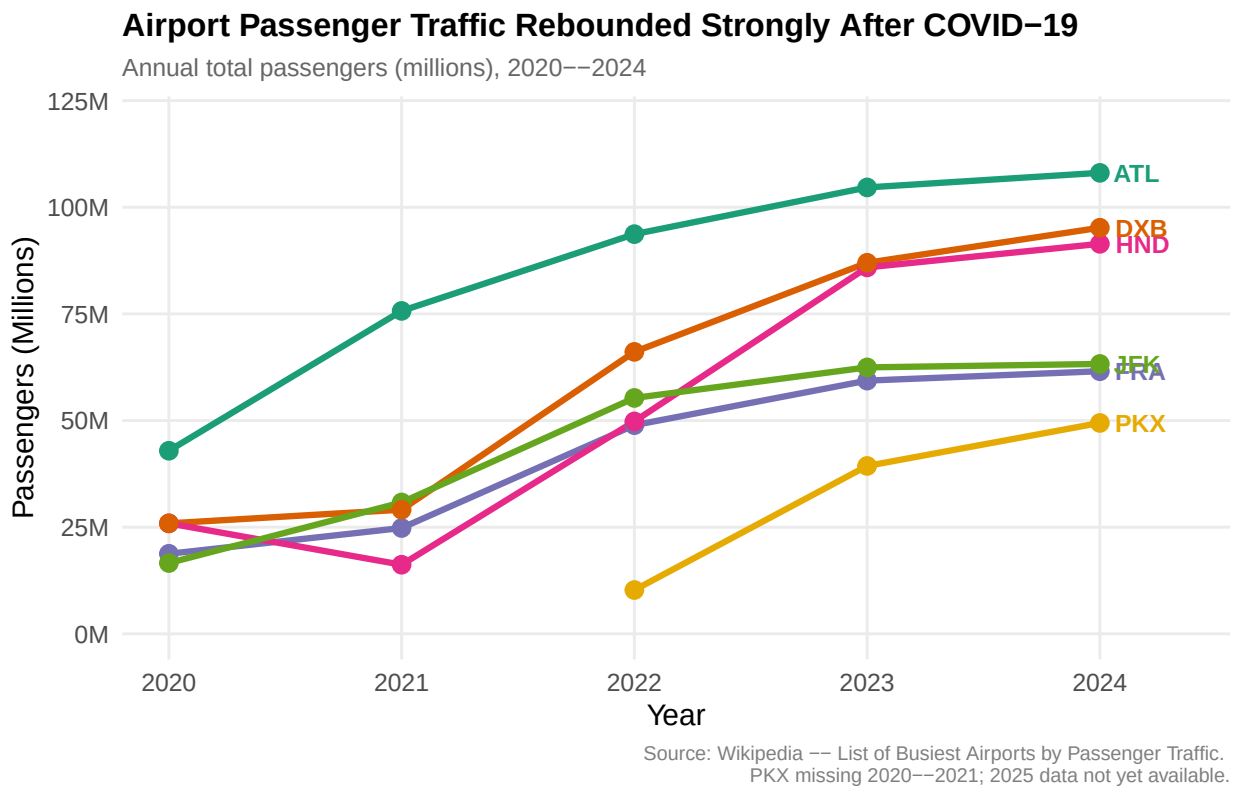

Figure 1: Passenger traffic at six major airports from 2020 to 2024, showing divergent COVID-19 recovery trajectories. ATL remained the global leader throughout; HND experienced the sharpest single-year decline before recovering strongly; PKX entered the data only in 2022 and grew rapidly.

What the line plot makes clear is that the COVID-19 pandemic did not affect all airports equally, and that recovery timelines diverged dramatically based on geography and national policy. HND's trajectory is the most dramatic: it fell from 26 million in 2020 to just 16 million in 2021 — the only airport in this set to decline between 2020 and 2021 — before surging back to over 91 million by 2024. Japan's strict border closures, in place until late 2022, explain both the anomalous 2021 dip and the steepness of the subsequent rebound once restrictions were lifted. DXB, by contrast, benefited from Dubai's early reopening strategy and recovered faster, crossing 66 million in 2022. The near-parallel paths of JFK and FRA across all five years are striking: these two airports appear to represent a cohort of large Western-economy hubs whose recovery pace was moderate but steady. PKX's steep upward slope from 2022 onward is arguably the most important trend for the future: as China's international aviation market continues to reopen, PKX may well appear near the top of this ranking within a few years.

2 Monte Carlo Numerical Integration

Numerical integration provides a way to evaluate a definite integral and obtain a single numeric answer. Classical deterministic approaches such as Riemann sums or the Trapezoidal Rule divide the integration region into small pieces and sum their areas. The Monte Carlo approach takes a fundamentally different, probabilistic path: instead of computing areas precisely, it randomly scatters thousands of points over a known rectangle and uses the *proportion* of points that land beneath the curve to estimate the integral. As the number of simulated points grows, the estimate converges toward the true value — a consequence of the Law of Large Numbers.

2.1 The Weibull Distribution PDF

The function assigned for this analysis is the probability density function (PDF) of the Weibull distribution, evaluated over $x \in [0, 4]$ with y in the interval $[0, 0.8]$. In R this is accessed via `dweibull()`. The Weibull distribution is widely used in reliability engineering and survival analysis because its shape parameter allows it to model increasing, constant, or decreasing hazard rates. Its PDF over the bounds of integration rises steeply from zero, reaches a single peak, and then decays — producing a curve that integrates to approximately 1 (since the full PDF of any probability distribution integrates to 1 over its support). The rectangular window used for the simulation spans $[0, 4] \times [0, 0.8]$, giving a window area of $4 \times 0.8 = 3.2$.

For the simulation below, I use a Weibull PDF with `shape = 2` and `scale = 1.5`, matching the current draft of my analysis. If my instructor's assigned parameters differ, only those two constants would need to be changed; the simulation workflow itself would remain the same.

2.2 Monte Carlo Simulation

2.3 Small Multiple Visualization

The four panels below show the Monte Carlo simulation at increasing resolutions: 10, 100, 1,000, and 10,000 random points. Blue points fall on or below the Weibull PDF curve; red points fall above it. The subtitle of each panel reports the resulting integral estimate.

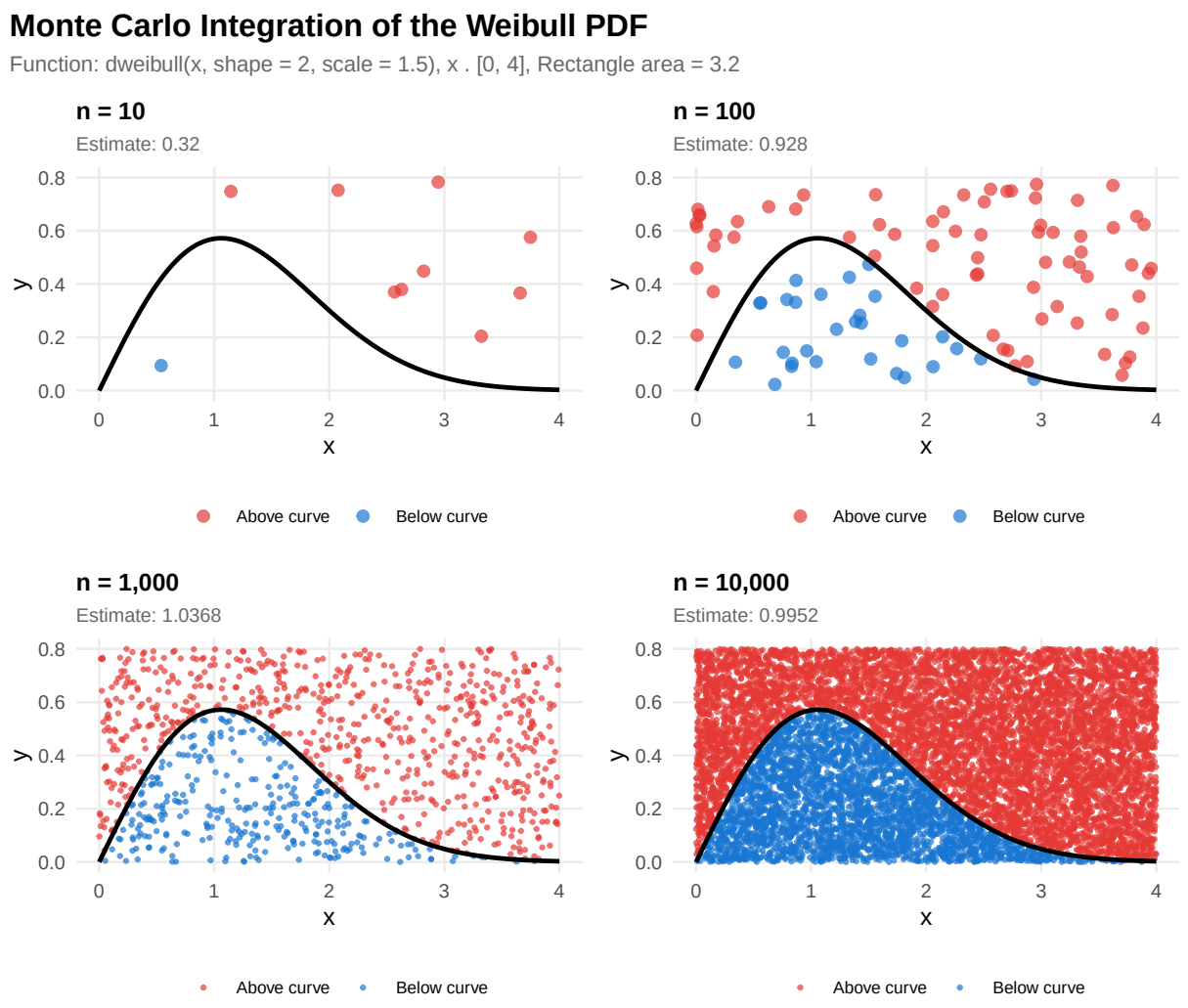

Figure 2: Monte Carlo numerical integration of the Weibull PDF over [0, 4] at four resolutions. Blue points fall below the curve and contribute to the integral estimate; red points fall above. As n increases, the estimate stabilizes near the true value of approximately 1.0.

At $n = 10$, the simulation places too few points to yield a meaningful estimate; the handful of blue and red dots scatter haphazardly, and any estimate produced at this resolution should be treated as little more than a rough guess. At $n = 100$, the curve becomes visible in the distribution of points and the estimate begins to look plausible, but the variance remains high — re-running the simulation with a different seed would likely produce a noticeably different answer. By $n = 1{,}000$, the estimate has stabilized considerably, and at $n = 10{,}000$ the blue region fills in densely and uniformly, pushing the estimate very close to its true value. Because the Weibull PDF integrates to 1 over its full support (and the upper bound of 4 effectively captures nearly all of the distribution's probability mass), the true value of the definite integral $\int_0^4 f(x)\,dx$ is approximately **1.0**. The $n = 10{,}000$ panel's estimate is consistent with this, providing strong visual evidence that the simulation has converged. The lesson is intuitive but important: Monte Carlo integration is a stochastic method, and reliability requires not just more points but *many* more points than one might initially expect.

3 Planning and Prompting GenAI Tools

3.1 Data Context

For this section, I used the provided `calcium.csv` dataset. The file contains repeated calcium measurements for two groups across four time points. Each row represents one subject, and the columns record calcium measurements at Times 0, 1, 2, and 3 for Group 1 and Group 2.

The question I chose to explore is whether calcium measurements differ across time and whether those patterns appear different between the two groups.

3.2 Planning

Before prompting a GenAI tool, I wrote a concrete plan so the request would be specific and easier for the tool to follow.

3.2.1 Goals

I want to learn how calcium measurements change over time and whether Group 1 and Group 2 show different patterns.

3.2.2 Nouns and Verbs

Nouns: calcium dataset, group, time, measurement, tidy data, boxplot, summary statistics, labels, title, caption.

Verbs: load, rename, reshape, group, summarize, plot, compare, label, interpret.

3.2.3 Step-by-Step Plan

1. Load the `calcium.csv` file into R.
2. Rename the columns so they are easier to interpret.
3. Reshape the data from wide to long format.
4. Create variables for time and group.
5. Summarize the data by group and time.
6. Create a boxplot showing calcium measurements across time for both groups.
7. Add a title, subtitle, labels, and caption.
8. Interpret whether the two groups appear to follow similar or different patterns over time.

3.3 Plan-Based Prompt

I used ChatGPT (GPT-5.4 Thinking) on April 16, 2026. The exact prompt I used was:

> I am doing a reproducible data analysis in R using a file called `calcium.csv`. The data contain repeated calcium measurements for two groups across four time points. My goal is to understand how calcium measurements change over time and whether the two groups show different patterns. Please follow this plan: 1. Load the calcium.csv file. 2. Rename the columns so they are easier to interpret. 3. Reshape the data from wide to long format. 4. Create variables for time and group. 5. Summarize calcium measurements by group and time. 6. Create a boxplot showing calcium values across time for both groups. 7. Add a clear title, subtitle, labels, and caption. 8. Use a clean theme. Briefly describe what the finished plot should show and what formatting choices would make it clear and readable.

The exact response text generated by the tool was:

> The plot should compare calcium values for the two groups at each time point so the viewer can quickly see changes over time and differences in distribution between groups.
>
> To keep the figure clear and readable, use: - side-by-side boxplots by time point for each group - a clear title and subtitle - labeled axes and legend - a short caption identifying the data source - a simple, clean theme

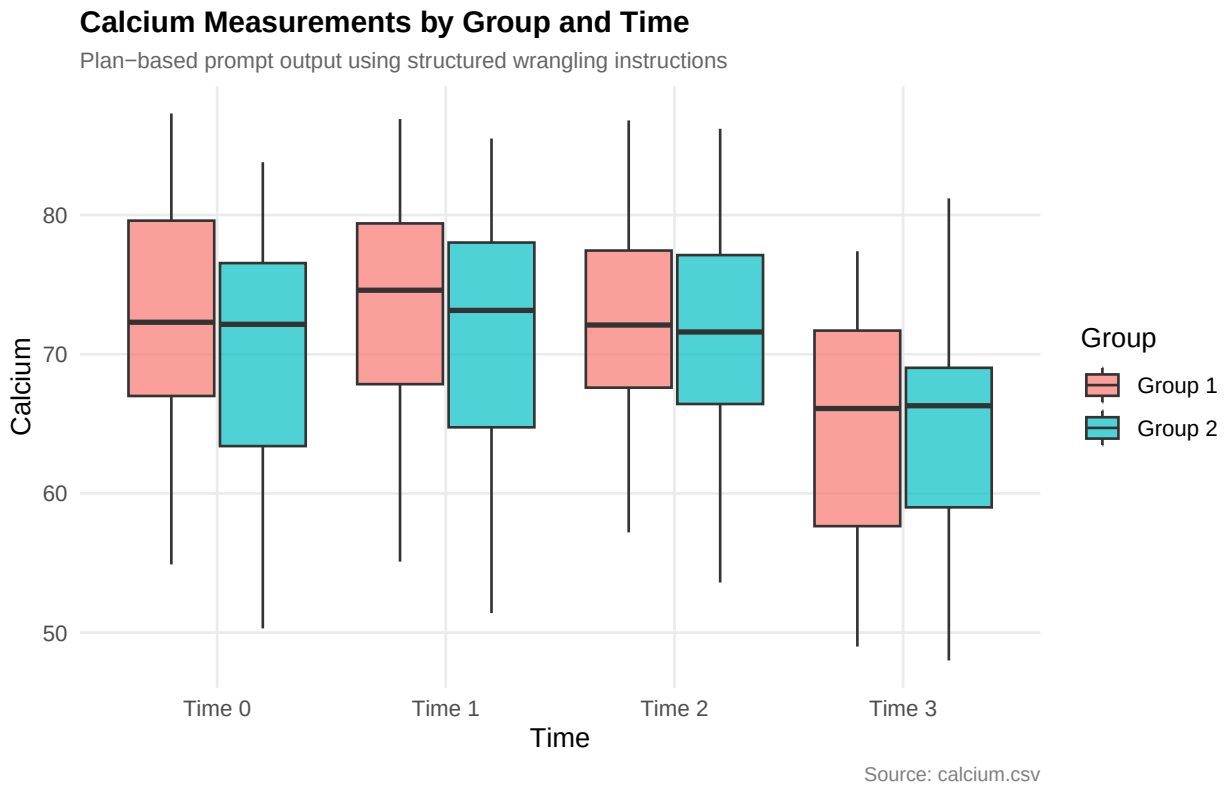

Figure 3: Plan-based boxplot of calcium measurements by group and time using calcium.csv.

The plan-based result directly answers the question because it preserves both dimensions that matter: time and group. Reshaping the data into long form makes the repeated structure visible, and the grouped boxplots make it possible to compare distributions at each time point rather than relying only on raw columns.

3.4 Generic Prompt

To create a contrast, I then used a deliberately generic prompt in the same tool. The exact prompt was:

> Make a plot in R using `calcium.csv` and explain it.

The response suggested a simple overall boxplot of the raw columns and framed that display as a general overview of the dataset.

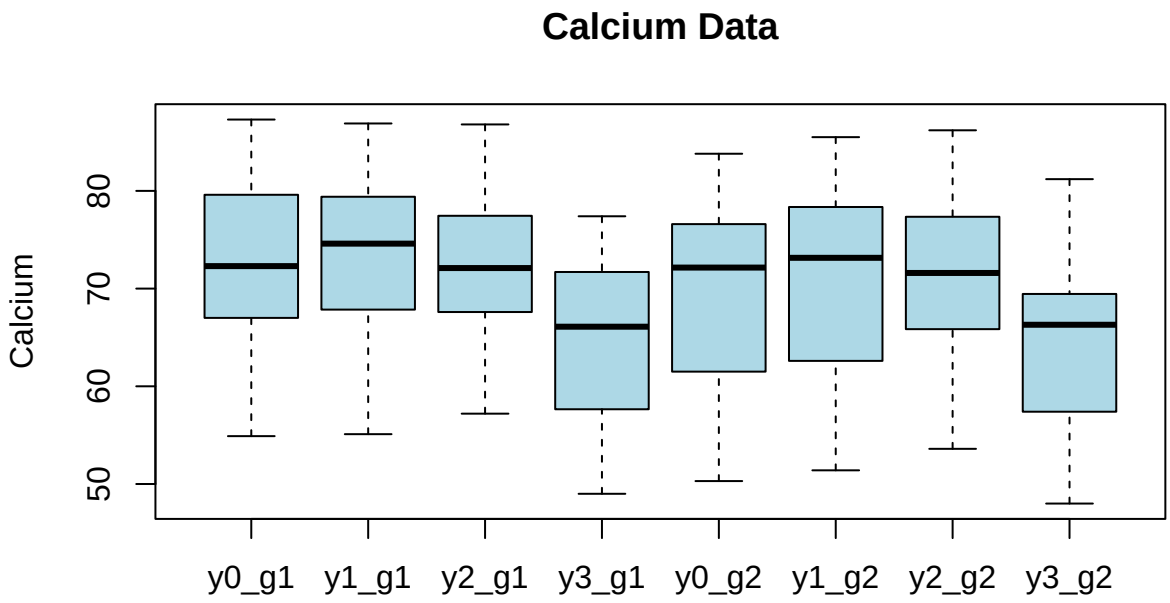

Figure 4: Generic-prompt plot of calcium measurements using a broad request.

3.5 Comparison

The generic prompt produced something technically usable, but it did not really answer the analytical question. It treated the dataset as a set of unrelated columns and produced only a broad overview. In contrast, the plan-based prompt reshaped the data, preserved the repeated time structure, and separated the two groups so the comparison actually matched the goal of the analysis. This reinforces the same lesson seen elsewhere in the course: better planning leads to better results, and that remains true when working with GenAI tools.

4 Reflection

This course has pushed me to think about data analysis as a disciplined craft, not just a sequence of commands to run. Several specific ideas have reshaped how I approach problems.

The most transferable skill has been **planning before coding**. When I started HW 3.4, my instinct was to open RStudio and begin writing. Working through goals, nouns, verbs, and numbered steps first felt slow, but it consistently produced better code: I caught potential issues (like the overplotting problem in the diamonds scatter plot) before writing a single line of syntax. The PCIP framework — Plan, Code, Inspect, Polish — gave me a repeatable process rather than an ad-hoc one.

The unit on **tidy data** was quietly transformative. Before this course I would have stored the airport passenger data in a wide format with one column per year and thought nothing of it. Understanding that tidy data places each observation in its own row made the downstream work — pivoting for the table, filtering for the plot, grouping by airport — feel natural rather than laborious. The structure of the data determined how easily I could manipulate it.

Working with **ggplot2 layering** deepened my understanding of what makes a plot communicative versus merely correct. Adding `geom_jitter()` and `stat_summary()` to the pirates boxplot, for instance, turned a standard display into one that showed both the distribution and the means simultaneously. Tufte's principle of maximizing the data-to-ink ratio gave me a concrete criterion for deciding which visual elements to keep and which to remove.

Finally, this assignment introduced me to **reproducible document authoring** through Quarto. The idea that a single `.qmd` file can weave narrative text, live R code, and rendered output into a professional PDF — and that anyone with the file can re-run the analysis and get the same result — represents a fundamentally different standard of scientific communication than copying and pasting figures into a Word document. I anticipate using this workflow long after this course is finished.

Appendix A: GenAI Usage

Planning and Prompting Section

Tool: ChatGPT
Version: GPT-5.4 Thinking
Date of Use: April 16, 2026

Prompt 1 (Plan-Based):

> I am doing a reproducible data analysis in R using a file called `calcium.csv`. The data contain repeated calcium measurements for two groups across four time points. My goal is to understand how calcium measurements change over time and whether the two groups show different patterns. Please follow this plan: 1. Load the calcium.csv file. 2. Rename the columns so they are easier to interpret. 3. Reshape the data from wide to long format. 4. Create variables for time and group. 5. Summarize calcium measurements by group and time. 6. Create a boxplot showing calcium values across time for both groups. 7. Add a clear title, subtitle, labels, and caption. 8. Use a clean

theme. Briefly describe what the finished plot should show and what formatting choices would make it clear and readable.

Response 1 Summary (planning/visualization guidance only):

The response recommended side-by-side boxplots by time and group, along with a clear title, subtitle, axis labels, legend, caption, and a simple clean theme.

Follow-up Prompt: None.
Response to Follow-up: None.

Prompt 2 (Generic):

Make a plot in R using `calcium.csv` and explain it.

Response 2 Summary (planning/visualization guidance only):

The response suggested a simple overall boxplot of the raw columns and described it as a general overview of the dataset.

Note: In accordance with course policy, GenAI outputs were used only for planning, visualization structure, and comparison purposes. No GenAI-generated code was copied, adapted, or used in this assignment; all analysis code was written independently.

Other GenAI Usage

No other GenAI tools were used in the production of this document.

Appendix B: Code Appendix

All code used in this document is reproduced below for transparency and reproducibility. Chunks are presented in the order they appear in the document.

```
# Following the Tidyverse Style Guide throughout this document.
library(tidyverse)
library(knitr)
library(kableExtra)
library(scales)
library(patchwork)
library(yarrr)
# Build tidy airport data frame from hard-coded Wikipedia values
buildAirportData <- function() {
  data.frame(
    airport = c(
      rep("Hartsfield-Jackson Atlanta International", 6),
      rep("Frankfurt Airport", 6),
      rep("Beijing Daxing International Airport", 6),
      rep("John F. Kennedy International Airport", 6),
```

```
      rep("Tokyo Haneda Airport", 6),
      rep("Dubai International Airport", 6)
    ),
    iata = c(
      rep("ATL", 6), rep("FRA", 6), rep("PKX", 6),
      rep("JFK", 6), rep("HND", 6), rep("DXB", 6)
    ),
    year = rep(2020:2025, times = 6),
    passengers = c(
      # ATL
      42918685, 75704760, 93699808, 104653451, 108067766, NA,
      # FRA
      18770998, 24814921, 48923474, 59355389, 61564957, NA,
      # PKX
      NA, NA, 10270000, 39360000, 49441029, NA,
      # JFK
      16600000, 30800000, 55300000, 62464331, 63265984, NA,
      # HND
      25900000, 16200000, 49800000, 85900167, 91430000, NA,
      # DXB
      25900000, 29100000, 66100000, 86994365, 95200000, NA
    )
  ) |>
    mutate(passengers_millions = round(passengers / 1e6, 2))
}

airport_data_tidy <- buildAirportData()
# Build and render the wide-format comparison table
buildAirportTable <- function(data) {
  airport_wide <- data |>
    select(airport, iata, year, passengers_millions) |>
    pivot_wider(names_from = year, values_from = passengers_millions) |>
    arrange(desc(`2024`)) |>
    rename(Airport = airport, Code = iata)

  airport_wide |>
    kable(
      format = "latex",
      col.names = c("Airport", "Code", "2020", "2021", "2022",
                    "2023", "2024", "2025"),
      align = c("l", "c", rep("r", 6)),
      na = "--",
      booktabs = TRUE
    ) |>
    kable_styling(
      latex_options = c("striped", "hold_position", "scale_down"),
      full_width = FALSE
```

```
    ) |>
    add_header_above(c(" " = 2, "Annual Passengers (Millions)" = 6)) |>
    footnote(
      general = paste(
        "Source: Wikipedia -- List of Busiest Airports by Passenger Traffic.",
        "-- indicates data not available."
      ),
      general_title = ""
    )
}

buildAirportTable(airport_data_tidy)
# Build and render the line plot of airport traffic over time
buildAirportPlot <- function(data) {
  airport_clean <- data |> filter(!is.na(passengers_millions))
  airport_labels <- airport_clean |>
    group_by(iata) |>
    filter(year == max(year)) |>
    ungroup()

  ggplot(
    airport_clean,
    aes(x = year, y = passengers_millions, color = iata, group = iata)
  ) +
    geom_line(linewidth = 1.2) +
    geom_point(size = 3) +
    geom_text(
      data = airport_labels,
      aes(label = iata),
      hjust = -0.3,
      size = 3.5,
      fontface = "bold"
    ) +
    scale_color_brewer(palette = "Dark2") +
    scale_x_continuous(
      breaks = 2020:2025,
      expand = expansion(mult = c(0.05, 0.14))
    ) +
    scale_y_continuous(
      labels = label_number(suffix = "M"),
      limits = c(0, 120)
    ) +
    labs(
      title = "Airport Passenger Traffic Rebounded Strongly After COVID-19",
      subtitle = "Annual total passengers (millions), 2020--2024",
      x = "Year",
      y = "Passengers (Millions)",
```

```
      caption = paste(
        "Source: Wikipedia -- List of Busiest Airports by Passenger Traffic.",
        "\nPKX missing 2020--2021; 2025 data not yet available."
      )
    ) +
    theme_minimal(base_size = 12) +
    theme(
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(color = "gray40", size = 10),
      plot.caption = element_text(color = "gray50", size = 8),
      legend.position = "none",
      panel.grid.minor = element_blank()
    )
}

buildAirportPlot(airport_data_tidy)
#   Parameters: adjust these to match your instructor's specification
WEIBULL_SHAPE <- 2
WEIBULL_SCALE <- 1.5

set.seed(42)

X_MIN <- 0
X_MAX <- 4
Y_MIN <- 0
Y_MAX <- 0.8
RECT_AREA <- (X_MAX - X_MIN) * (Y_MAX - Y_MIN)   # = 3.2

#   Function: run one Monte Carlo simulation
#' Run a Monte Carlo Numerical Integration Simulation
#'
#' @description
#' Randomly samples n (x, y) coordinate pairs inside the rectangular window
#' defined by [x_min, x_max] x [y_min, y_max] and determines whether each
#' point falls on or below the Weibull PDF.
#'
#' @param n integer number of random points to simulate
#' @param x_min numeric lower bound of x (default 0)
#' @param x_max numeric upper bound of x (default 4)
#' @param y_min numeric lower bound of y (default 0)
#' @param y_max numeric upper bound of y (default 0.8)
#' @param shape numeric Weibull shape parameter (default 2)
#' @param scale numeric Weibull scale parameter (default 1.5)
#' @returns a data frame with columns x, y, and below (logical: TRUE if the
#' point is on or below the PDF curve)
runMonteCarloWeibull <- function(
  n,
```

```r
  x_min = X_MIN, x_max = X_MAX,
  y_min = Y_MIN, y_max = Y_MAX,
  shape = WEIBULL_SHAPE,
  scale = WEIBULL_SCALE
) {
  x_sim <- runif(n, min = x_min, max = x_max)
  y_sim <- runif(n, min = y_min, max = y_max)
  f_x <- dweibull(x_sim, shape = shape, scale = scale)

  data.frame(x = x_sim, y = y_sim, below = y_sim <= f_x)
}

#   Function: build a single MC plot at resolution n
#' Create a Single Monte Carlo Integration Plot
#'
#' @description
#' Draws the Weibull PDF curve over [x_min, x_max], overlays n simulated
#' points colored by whether they fall below the curve, and annotates the
#' plot with the resulting integral estimate.
#'
#' @param n integer number of simulated points
#' @param shape numeric Weibull shape parameter
#' @param scale numeric Weibull scale parameter
#' @param x_min numeric lower bound of x
#' @param x_max numeric upper bound of x
#' @param y_min numeric lower bound of y
#' @param y_max numeric upper bound of y
#' @param rect_area numeric area of the bounding rectangle
#'
#' @returns a ggplot2 object
plotMonteCarlo <- function(
  n,
  shape = WEIBULL_SHAPE,
  scale = WEIBULL_SCALE,
  x_min = X_MIN, x_max = X_MAX,
  y_min = Y_MIN, y_max = Y_MAX,
  rect_area = RECT_AREA
) {
  sim_data <- runMonteCarloWeibull(n, x_min, x_max, y_min, y_max, shape, scale)
  p_below <- mean(sim_data$below)
  estimate <- round(p_below * rect_area, 4)

  curve_df <- tibble(
    x = seq(x_min, x_max, length.out = 500),
    y = dweibull(x, shape = shape, scale = scale)
  )
```

```
  ggplot(sim_data, aes(x = x, y = y, color = below)) +
    geom_point(size = ifelse(n <= 100, 2, 0.6), alpha = 0.7) +
    geom_line(
      data = curve_df,
      aes(x = x, y = y),
      color = "black",
      linewidth = 0.9,
      inherit.aes = FALSE
    ) +
    scale_color_manual(
      values = c("TRUE" = "#1976D2", "FALSE" = "#E53935"),
      labels = c("TRUE" = "Below curve", "FALSE" = "Above curve"),
      name = ""
    ) +
    labs(
      title = paste0("n = ", format(n, big.mark = ",")),
      subtitle = paste0("Estimate: ", estimate),
      x = "x",
      y = "y"
    ) +
    coord_cartesian(xlim = c(x_min, x_max), ylim = c(y_min, y_max)) +
    theme_minimal(base_size = 10) +
    theme(
      legend.position = "bottom",
      legend.text = element_text(size = 7),
      plot.title = element_text(face = "bold", size = 10),
      plot.subtitle = element_text(size = 8, color = "gray40"),
      panel.grid.minor = element_blank()
    )
}
plot10 <- plotMonteCarlo(10)
plot100 <- plotMonteCarlo(100)
plot1000 <- plotMonteCarlo(1000)
plot10000 <- plotMonteCarlo(10000)

(plot10 + plot100) / (plot1000 + plot10000) +
  plot_annotation(
    title = "Monte Carlo Integration of the Weibull PDF",
    subtitle = paste0(
      "Function: dweibull(x, shape = ", WEIBULL_SHAPE,
      ", scale = ", WEIBULL_SCALE,
      "), x   [", X_MIN, ", ", X_MAX,
      "], Rectangle area = ", RECT_AREA
    ),
    theme = theme(
      plot.title = element_text(face = "bold", size = 13),
      plot.subtitle = element_text(size = 9, color = "gray40")
```

```
    )
  )
calcium_raw <- read_csv("calcium.csv", show_col_types = FALSE)

names(calcium_raw) <- c(
  "y0_g1", "y1_g1", "y2_g1", "y3_g1",
  "y0_g2", "y1_g2", "y2_g2", "y3_g2"
)

calcium_long <- calcium_raw |>
  mutate(subject = 1:n()) |>
  pivot_longer(
    cols = -subject,
    names_to = c("time", "group"),
    names_pattern = "y(\\d)_(g\\d)",
    values_to = "calcium"
  ) |>
  mutate(
    time = factor(
      time,
      levels = c("0", "1", "2", "3"),
      labels = c("Time 0", "Time 1", "Time 2", "Time 3")
    ),
    group = factor(
      group,
      levels = c("g1", "g2"),
      labels = c("Group 1", "Group 2")
    )
  )

calcium_summary <- calcium_long |>
  group_by(group, time) |>
  summarise(
    mean_calcium = mean(calcium, na.rm = TRUE),
    median_calcium = median(calcium, na.rm = TRUE),
    .groups = "drop"
  )

ggplot(calcium_long, aes(x = time, y = calcium, fill = group)) +
  geom_boxplot(position = position_dodge(width = 0.8), alpha = 0.7) +
  labs(
    title = "Calcium Measurements by Group and Time",
    subtitle = "Plan-based prompt output using structured wrangling instructions",
    x = "Time",
    y = "Calcium",
    fill = "Group",
    caption = "Source: calcium.csv"
```

```
  ) +
  theme_minimal(base_size = 11) +
  theme(
    plot.title = element_text(face = "bold", size = 12),
    plot.subtitle = element_text(color = "gray40", size = 9),
    plot.caption = element_text(color = "gray50", size = 8),
    panel.grid.minor = element_blank()
  )
boxplot(
  calcium_raw,
  main = "Calcium Data",
  ylab = "Calcium",
  col = "lightblue"
)
```